Optimized and kinematically feasible multi-agent motion planning

Anja Hellander[1], Kristoffer Bergman[2] and Daniel Axehill[1]

***Abstract*— Multi-agent motion planning (MAMP) is an important problem for autonomous systems with multiple agents. In this work we propose a two-step method for finding optimized and kinematically feasible solutions to MAMP problems. The first step finds an initial feasible solution using state-of-the-art methods such as conflict-based search (CBS) or priority-based search (PBS), and the second step is an improvement step which improves the solution by solving a multi-phase optimal control problem (OCP) where the initial solution is used to warm-start the solver. We also propose a method for generating motion primitives in an optimized way under the constraint that the primitive durations are all multiples of the same sample time.**

We evaluate our proposed framework on a MAMP problem for tractor-trailer systems. We extend the safe interval path planning with interval projections (SIPP-IP) algorithm so it can handle more general cost functions and larger agents, but our results show that for the tractor-trailer system a simple lattice-based planner performs better due to less conservative collision checks. Our experiments also indicate that CBS performs better than PBS for this system as it achieves a higher success rate in environments with obstacles and had a lower average runtime, although both planners achieve solutions of similar quality after the improvement step.

I. Introduction

An important problem to solve for autonomous systems is motion planning, i.e., determining what path or trajectory to follow in order to safely and efficiently move the system from a current state to a desired state. An even more challenging problem is multi-agent motion planning (MAMP), where feasible and collision-free trajectories for multiple agents should be planned. In this work, we focus on optimized and kinematically feasible MAMP for agents with complex kinematics such as tractor-trailers.

A. Related Work

A simpler version of MAMP is the multi-agent pathfinding (MAPF) problem [1]–[3], where time is discretized and kinodynamic constraints on the agents are not considered. In the most simple form of MAPF, agents move on a grid where each agent occupies one grid cell and at each discretized-step agents will either stay at their current cell or move to a neighbouring cell. More generally, agents move on a graph where vertices represent possible agent states and edges represent possible motions between states.

*This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

[1]A. Hellander and D. Axehill are with the Division of Automatic Control, Linköping University, Sweden {anja.hellander, daniel.axehill}@liu.se

[2]K. Bergman is with Saab AB, Linköping, Sweden kristoffer.bergman@thernfrst.io

The optimal MAPF problem, where either the sum or the maximum of the agent times is minimized, is NP-complete [4]. The most common approach to optimal MAPF is conflict-based search (CBS) [5], [6], where each agent initially finds a trajectory independently of other agents, and constraints on the trajectories are added until no conflicts between trajectories remain. Other approaches include extending the A* algorithm [7] to search in a multi-agent search space [8]–[10], and the increasing cost tree search (ICTS) that uses a bilevel search [11], [12].

As solving the optimal problem is difficult, in practice suboptimal algorithms are used, such as Prioritized Planning (PP) [13]–[15] where a path is found for one agent at a time according to a pre-set priority order, and paths of agents with higher priority are seen as dynamic obstacles. An improvement to this is Priority Based Search (PBS) [16], where several partial priority orders are explored. While PP and PBS are often efficient, they are neither optimal nor complete, even if all possible priority orders are explored [16].

There has been an increased interest in extending the MAPF algorithm so as to be able to solve more complex problems that are more similar to MAMP. Classical MAPF assumes that agents consist of only a single point and therefore occupies only a single grid cell or vertex. A problem for train-like agents is considered in [17], where agents cover $k+1$ grid cells: the grid cell currently occupied by the head of the train as well as the last k positions occupied by the head which are now occupied by the train. MAPF for so-called large agents, that have an associated shape and may occupy more than a single grid cell, is considered in [18]. However, only agent shapes such that the area occupied is the same regardless of orientation such as circles are considered.

MAPF with continuous time and agent motions with non-unit cost that obey some kinematic constraints is considered in [19]. In addition, the algorithm presented in [20] allows for an arbitrary wait time. Both algorithms rely on the Safe Interval Path Planning (SIPP) [21] algorithm for single-agent motion planning. SIPP, however, makes the assumption that agents can always perform a wait action (i.e., remain at its current grid cell or vertex). In real-life scenarios, agents are typically unable to instantaneously stop or go from standstill to maximum velocity. The trajectories obtained are therefore either not kinodynamically feasible, or they are such that the agent is at standstill at all states corresponding to vertices in the graph in order to allow for wait actions, which leads to suboptimal trajectories. In [22] an extension to SIPP called Safe Interval Path Planning with Interval Projection (SIPP-IP) was presented. Unlike SIPP, SIPP-IP only allows wait

actions to be performed if the velocity of the agent is zero.

Kinodynamic constraints have also been considered in [23], [24]. In [23], MAMP for car-like vehicles is performed by using CBS in combination with a spatiotemporal Hybrid A* (SHA*). The resulting trajectories are, however, not truly feasible as they assume, e.g., that the vehicles can instantaneously switch between standstill, forward driving at a constant velocity and driving reverse at a constant velocity. A similar approach is taken in [25], but for small tractor-trailer vehicles and an additional smoothing step is applied to make the trajectories feasible. In [26], CBS is used in combination with probabilistic roadmaps (PRM). Similarly, [24] uses CBS, but uses rapidly-exploring random trees (RRT) [27], [28] as single-agent motion planner and hence the algorithm lacks optimality as well as resolution optimality.

There are several approaches to optimal motion planning. A common approach to motion planning is to use deterministic or random sampling-based motion planners. Many such algorithms that use random sampling are based on the Rapidly-exploring Random Tree (RRT) algorithm [27] and its asymptotically optimal extension RRT* [29]. There are extensions of RRT* that can handle dynamical and nonholonomical constraints [30], [31], but the algorithms require the solution of optimal control problems (OCPs) in order to connect states in the generated search tree, which can be computationally demanding for general nonlinear systems [31].

For differentially flat systems, B-splines or Bézier curves are commonly used to smoothen a sequence of previously computed waypoints [32], or as steering functions used within a sampling-based motion planner.

The optimal motion-planning problem can also be formulated as an OCP directly, either as a mixed-integer problem or as a nonlinear problem. In practice, a numerical solver must often be warm-started with a good initial guess to reach convergence. For this reason, one approach is to solve the motion planning in steps where an initial solution is first computed and then used to warm-start the numerical solver. In [33] a rough, possibly infeasible, initial trajectory for a tractor-trailer vehicle is computed using Hybrid A* and used as warm-start to solve a sequence of OCPs that gradually add constraints to ensure kinematic feasibility, obstacle avoidance etc. In [34] a lattice-based motion planner [35] is used to compute an initial feasible solution which is then used as warm-start to solve an OCP. Similarly, [36] uses a lattice-based planner to compute an initial path, for which a time-optimal velocity profile is then computed.

Similar optimization approaches have also been applied to MAMP problems. In [37] initial paths for all agents are computed which are then used to compute reference trajectories that are used as input to a mixed-integer quadratic program formulation of model predictive control. This approach, however, is only applicable if the agents have linear (or piecewise affine) dynamics. In [38] reference trajectories that are not guaranteed to be neither dynamically feasible nor collision-free are first computed. These are then smoothed and made to be collision-free by solving an optimization problem within a sequential programming framework.

B. Contributions

We propose a two-step MAMP approach where in a first step an initial feasible solution is computed using either CBS or PBS, and in a second step a locally optimized solution is computed by posing and solving a multi-phase optimal control problem using the initial feasible solution as initial guess to warm start the numerical solver.

Furthermore, we introduce modifications to SIPP-IP to enable more general cost functions and larger agents. Although SIPP-IP has been shown to outperform a lattice-based A* search as single-agent planner, we show that for complex MAMP problems such as the one for tractor-trailer systems, the lattice-based A* search can perform better.

Finally, we propose a method for generating optimal kinematically feasible motion primitives that are synchronized in time, i.e., have time durations that are a multiple of the same time step length. This is useful for collision checking and is also a requirement for SIPP-IP [22].

II. Problem Formulation

Consider k agents operating within the same workspace $W \subset \mathcal{R}^2$. Each agent $i \in \{1, \dots, k\}$ is a nonlinear dynamical system that can be described by

$$\dot{x}_i(t) = f_i(x_i(t), u_i(t)), \quad x(t_0) = x_{i,0} \tag{1}$$

where $x_i \in \mathcal{X}_i \subset \mathcal{R}^{n_i}$ is the state vector, $u_i \in \mathcal{U}_i \subset \mathcal{R}^{m_i}$ is the control input, $t \geq 0$ is the time elapsed, and $x_{i,\text{init}}$ is the initial state of the agent. Let $R_i(x_i) \subset \mathcal{R}^2$ denote the area occupied by agent i when the agent is in state x_i. Then, $\mathcal{X}_i$ is the state space for agent i corresponding to $\mathcal{X}_i = \{x_i \in \mathcal{R}^{n_i} | R_i(x_i) \subset W\}$.

Furthermore, there is an obstacle region $O \in W$ that the agents should not collide with. Thus, each agent i is further constrained as

$$x_i(t) \in \mathcal{X}_i \setminus \mathcal{X}_{i,\text{obs}} = \mathcal{X}_{i,\text{free}} \tag{2}$$

where $\mathcal{X}_{i,\text{obs}}$ denotes the part of the state space that corresponds to the agent intersecting with the obstacle region O, i.e., $\mathcal{X}_{i,\text{obs}} = \{x_i \in \mathcal{X}_i | R_i(x) \cap O \neq \emptyset\}$. Additionally, agents must not collide with each other, i.e., they must obey

$$R_i(x_i(t)) \cap R_j(x_j(t)) = \emptyset \quad i \neq j \tag{3}$$

Given the initial states $x_{i,0}$ and final states $x_{i,f}$ for each agent, the multi-agent motion-planning problem consists of finding a set of feasible and collision-free trajectories $(x_i(t), u_i(t), t_{i,f})$ such that $x_i(t_0) = x_{i,0}$ and $x_i(t_{i,f}) = x_{i,f}$ for all agents i.

The optimal MAMP problem for k agents can then be posed as

$$\underset{\{x_i(t),u_i(t),t_{i,f}\}_{i=1}^{k}}{\text{minimize}} \quad \sum_{i=1}^{k} J_i(x_i,u_i,t_{i,f}) \tag{4a}$$

$$\text{s.t.} \quad \dot{x}_i(t) = f_i(x_i(t),u_i(t)) \tag{4b}$$

$$x_i(t) \in \mathcal{X}_{i,\text{free}} \quad i=1,\dots,k \tag{4c}$$

$$u_i(t) \in \mathcal{U}_i \tag{4d}$$

$$R_i(x_i(t)) \cap R_j(x_j(t)) = \emptyset \quad i \neq j \tag{4e}$$

$$x_i(t_0) = x_{i,0} \tag{4f}$$

$$x_i(t_{i,f}) = x_{i,f} \tag{4g}$$

where the cost functions J_i are defined as

$$J_i = \int_{t_0}^{t_f} l(x_i(t),u_i(t))\,dt \tag{5}$$

where $l(x(t),u(t)) \geq 0$ is the user-defined running cost. For example, selecting $l(x,u) = 1$ results in a minimum-time problem.

Finding a feasible and globally optimal solution to (4) is difficult, even for $k = 1$, due to the combinatorial aspects of what route to take around obstacles as well as the nonlinearity of the system. For $k > 1$ there is also the combinatorial problem of which agent should be given priority if their paths overlap. For this reason, it is common to use methods that find feasible solutions by solving approximate problems [39]. One such approach for single-agent motion planning is the lattice-based motion planner [35].

Finding a feasible and locally optimal solution is even more difficult if $k > 1$. If not for constraint (4e), it would be possible to treat (4) as k separate single-agent motion-planning problems. It is common to use methods that find feasible solutions by repeatedly solving (approximate) single-agent motion-planning problems and gradually introduce constraints to ascertain that no collision between agents occurs. Examples of such algorithms include CBS [5] and PBS [16].

A. The discretized problem

Lattice-based motion planners [35] rely on restricting the control signals to a discrete subset of so-called motion primitives, thereby transforming (4) from a continuous optimization problem to a discrete graph-search problem on a graph $\mathcal{G} = (\mathcal{V},\mathcal{E})$.

To do so, a discretized state space $\mathcal{X}_d$ is constructed by sampling the state space $\mathcal{X}$ in a regular fashion. Next, the connectivity is selected by deciding which pair of states in $\mathcal{X}_d$ to connect. For each such pair a motion primitive describing a feasible motion (assuming no obstacles) between the states is computed, e.g., by solving an OCP [40]. A motion primitive m is defined as

$$m = (x(t),u(t)) \in \mathcal{X} \times \mathcal{U}, t \in [0,T] \tag{6}$$

where T is the time duration of the primitive and $x(0), x(T) \in \mathcal{X}_d$.

The single-agent motion-planning problem can then be approximated as the discrete OCP

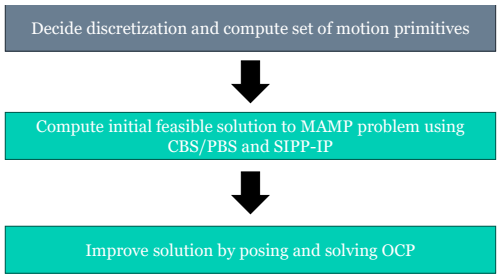

Fig. 1. The steps of the proposed framework. Steps in grey are performed off-line and steps in green are performed on-line.

$$\underset{\{m_i\}_{i=1}^{N},N}{\text{minimize}} \quad J_{\text{lat}} = \sum_{i=1}^{N} L_m(m_i) \tag{7a}$$

$$\text{s.t.} \quad x_1 = x_0 \tag{7b}$$

$$x_{N+1} = x_f \tag{7c}$$

$$t_1 = t_0 \tag{7d}$$

$$x_{i+1} = \mathbf{f_m}(x_i,m_i) \tag{7e}$$

$$t_{i+1} = t_i + T_{m_k} \tag{7f}$$

$$m_k \in \mathcal{P} \tag{7g}$$

$$R(x_i,m_i,t) \notin \mathcal{X}_{\text{obs}}(t) \quad t \in [t_i,t_{i+1}] \tag{7h}$$

where $\mathbf{f_m}(x,m)$ describes the successor state after motion primitive $\mathbf{m}$ is applied in $\mathbf{x}$.

Successful state-of-the-art multi-agent planners such as CBS and PBS that rely on decomposing the problem into many single-agent planning problems can then solve the approximated problem (7).

III. The Planning Framework

The proposed MAMP framework can be seen as an extension of the single-agent framework proposed in [34]. The underlying principle is to first solve a discretization of the motion-planning problem to obtain an initial feasible solution. As shown in [34], solving the discretized problem will in general result in a suboptimal solution. For this reason, an improvement step is applied where an OCP is posed and solved using the initial feasible solution found in the previous step as initial guess to warm-start the solver. The steps of the algorithm are shown in Figure 1.

IV. Computing Time-Synchronous Motion Primitives

To enable simple inter-agent collision checking, it is necessary that all motion primitives (represented as a sequence of samples) to share the same discretization, i.e., the time between samples along the motion primitive should be the same for all motion primitives. To achieve this, we propose here a novel extension of the framework in [41].

Optimal motion primitives are generated by first discretizing the state space, then selecting the connectivity, i.e.,

Algorithm 1 Primitive generation

1: Input: initial state x_0, primitive maneuver q, sample time t_s
2: minCost = ∞, primitive = $\emptyset$
3: **for** $x_T \in$ candidateTerminalStates(x_0, q) **do**
4: p = optimizePrimitive(x_0, x_T)
5: T = duration(p)
6: **for** $N \in \{\left\lfloor \frac{T}{t_s} \right\rfloor, t_s \left\lceil \frac{T}{t_s} \right\rceil\}$ **do**
7: $\bar{p} =$ optimizePrimitive(x_0, x_T, N, t_s)
8: **if** cost$(\bar{p}) <$ minCost **then**
9: minCost = cost$(\bar{p})$
10: primitive = $\bar{p}$
11: **end if**
12: **end for**
13: **end for**
14: **return** primitive

deciding which discretized states to connect, and finally solving OCPs to determine a feasible trajectory between the states to connect. As in [34], we use the same cost function (5) for both the primitive generation and the subsequent motion planning.

To compute the primitives, we use an extended version of the framework from [41] where different primitive maneuvers such as parallel or straight are selected and the connectivity is then automatically selected for each maneuver. In the original framework the sample time used by each primitive is a decision variable and is allowed to vary between the primitives. We modify the framework so that we first select a sample time t_s to be used for all primitives. We then select the connectivity and length of the primitives as outlined in Algorithm 1.

For each pair of initial state and primitive maneuver a set of candidate terminal states are generated. For each candidate terminal state a motion primitive is generated by solving the corresponding OCP without any constraint on the sample time (line 4). Based on the time duration T of the motion primitive two candidate primitive durations are computed as $t_s \left\lfloor \frac{T}{t_s} \right\rfloor$ and $t_s \left\lceil \frac{T}{t_s} \right\rceil$ respectively. For each candidate duration the OCP is solved again with an additional constraint on the time duration and the sample time (line 7), resulting in a candidate motion primitive. The candidate primitive with the best cost function value is then selected.

V. Computing an Initial Feasible Solution

This section describes the framework for computing an initial feasible solution.

A. Multi-agent planner

As multi-agent motion planner we consider both CBS and PBS. CBS and PBS are both based on searching a graph where nodes contain a set of constraints on the trajectories of each agent as well kinematically feasible trajectories that obey these constraints. A general outline is shown in Algorithm 2. First, the root node is created without any

Algorithm 2 General MAMP algorithm

1: **procedure** MAMP
2: Root.constraints = $\emptyset$
3: Root.plan = updatePlan(Root, Root.constraints)
4: Q.insert(Root)
5: **while** not Q.empty() **do**
6: $n = Q$.pop()
7: **if** n.conflicts() $= 0$ **then**
8: **return** n.plan
9: **end if**
10: $C = n$.firstConflict()
11: **for** agent i involved in C **do**
12: $n' = n$
13: n'.constraints $=$ n.constraints $\cup$ makeConstraint(C, i)
14: n'.plan = updatePlan(n', n'.constraints)
15: Q.insert(n')
16: **end for**
17: **end while**
18: **end procedure**

constraints and a plan consisting of trajectories for each agent is computed (Line 3) by ignoring constraint (7b) in (7). Next, nodes are iteratively removed from the open list (Line 6) and checked for conflicts, i.e., collisions between agents. A conflict between a pair of agents a_i, a_j is selected (Line 10) and two new nodes are created to which constraints on the trajectories of a_i and a_j, respectively, are added so as to solve the conflict (Line 13). The trajectories of the agents are updated so that the constraints are obeyed (Line 14) and the new nodes are inserted into the open list.

In CBS, each constraint that is added applies only at a given point in time. Depending on implementation, constraints may specify that an agent is not allowed to be in a certain state at a certain time, or is not allowed to apply a certain motion primitive from a certain state at a certain time.

In PBS, constraints are in the form of partial priority orders that specify which agents take priority. Agents with lower priority treat agents with higher priority as dynamic obstacles and plan trajectories that avoid them.

B. Single-agent motion planner

As single-agent motion planner we propose to use a lattice-based motion planner where the state has been augmented with time.

We also extended SIPP-IP to allow for general cost functions J_i on the form (5). In the original SIPP-IP, each search node corresponds to an agent state and an associated time interval $[t_l, t_u]$ and the cost-to-come of a search node is $g(n) = t_l$. To perform the desired extension, we assume that each motion primitive is associated with a cost and that the cost-to-come of a node is $g(n) = t_l + \bar{g}(n)$, where $\bar{g}(n)$ is the sum of the costs of the motion primitives necessary to reach n. The proposed extension introduces more book-keeping to check if a node has been seen before with lower cost or

not. In the original algorithm it is sufficient to store the time intervals that have been explored for each state. If a new time interval $[t_l, t_u]$ is explored where another time interval $[t_l, t]$ has already been explored, only the remaining interval $[t, t_u]$ will be considered. With the proposed modifications, however, it is possible to reach a state at a time that overlaps with an already explored time interval, but with a lower cost. For each explored time interval it is then necessary to keep track of the lowest cost-to-come seen for that interval. Only repeated time intervals with an equal or higher cost are then discarded.

SIPP-IP handles collisions by dividing the environment into a grid cell. Each motion primitive stores a list of what cells are swept during what intervals, and collision checking is performed by checking against the unsafe intervals of each grid cell. The original SIPP-IP implementation assumed that the agent occupied only a single grid cell at the end of a motion primitive. This is not realistic for large agents such as tractor-trailers. We therefore introduce the possibility to have larger agents by allowing multiple cells to be terminal cells. With only a single terminal cell it is straightforward to determine t_u for a state with zero velocity as it will be the upper bound of the safe interval for the terminal cell. When multiple terminal cells are allowed, as in this work, t_u is instead selected as the lowest such upper bound.

There are two major differences between the two single-agent motion-planning algorithms considered. First, a search node in SIPP-IP corresponds to an agent state and a time interval whereas a search node in the lattice-based planner corresponds to an agent state and a single point in time. This suggests that SIPP-IP is more efficient as fewer states are needed. The second difference is collision checking. SIPP-IP uses grid cells and intervals as described above. The lattice-based planner instead checks for collisions at each time step by approximating the agents and static obstacles with covering discs and performing circle-to-circle collision checks. These circles are used when computing the cells that are swept by the motion primitives for SIPP-IP as well. As a cell is considered occupied as long as any part of the circle is inside it this implies that SIPP-IP has a more conservative collision avoidance, which in general will result in plans with lower performance. In particular, two agents with circles sweeping through the same cell will be considered to collide even though the circles may not actually overlap.

VI. Improvement using Optimal Control

In this section we describe the improvement step, where we propose to use numerical optimal control to improve the initial feasible solution, in line with what has been proposed for single-agent motion-planning problems in [34].

Initially this might seem straightforward, but posing (4) as on overarching OCP that can be solved by off-the-shelf solvers turns out to be non-trivial. In the single-agent case, practical implementations typically rely on introducing variables $x_n, n = 0, 1, \ldots, N$ where $x_n = x(nT_s), T_s = \frac{t_f}{N}$ and the terminal time t_f is a decision variable in the optimization problem. This representation makes it easy to enforce the initial and terminal constraints (4f) and (4g) by constraining the values of x_0 and x_N, respectively. It is not immediately obvious how to extend this to a multi-agent setting where the terminal times $t_{i,f}$ are not necessarily the same. If on the one hand the trajectories $x_i(t)$ are discretized independently of each other and $t_{i,f}$ are introduced as decision variables it is not straightforward to encode the collision-avoidance constraint (4e) as it is not straightforward to determine which discretized variables that correspond to the same instant in absolute time as the $t_{i,f}$ change values during the optimization. On the other hand, if all trajectories use the same T_s, then it is straightforward to implement (4e), but difficult to implement (4g). That is, if $x_{i,N_i} = x_{i,f}$ is enforced, then there is little freedom in reducing $t_{i,f}$ as reducing the terminal time for one agent must result in the terminal times for all other agents being reduced in equal proportion. The alternative is to find some other way to encode (4g).

The solution we propose in this work is to pose the problem (4) as a multi-phase OCP. Using the trajectories $(x_i(t), u_i(t), t_{i,f})$ obtained in the first step we assume without loss of generality that the agents are ordered such that $t_{1,f} \leq t_{2,f} \leq \cdots \leq t_{k,f}$, i.e., that if $i < j$, then agent i reaches its goal no later than agent j. If not, we simply reorder them. We divide the problem into $M = k$ phases where the first phase covers the time until agent $i = 1$ reaches its goal and each subsequent phase m covers the time between the time when agent $m-1$ reaches its goal and agent m reaches its goal. An illustration of this is shown in Figure 2. For each phase m we introduce the decision variable t_m denoting the time duration of the phase. For each phase it is then possible to use the same time discretization for all agent trajectories to allow for easy collision checking, while different phases having different discretization allows for some agent trajectories to decrease in time while others increase as the time duration of each phase is independent of the others.

Note that the order in which the agents reach their goal will be the same as in the initial feasible solution from the first step. Similarly, the combinatorial aspect of how to navigate around static obstacles as well as other agents is also implicitly encoded by the initial feasible solution and will be maintained.

The OCP to solve can then be formulated as

$$\underset{\{x_i(t),u_i(t),t_i\}_{i=1}^{k}}{\text{minimize}} \quad \sum_{m=1}^{k} J_m(x,u,t_m) \tag{8a}$$

$$\text{s.t.} \quad x_{0,i}(t_0) = x_{i,0} \tag{8b}$$

$$x_{m,m}(t_m) = x_{m,f} \tag{8c}$$

$$x_{m+1,i}(t_m) = x_{m,i}(t_m), i \geq m+1 \tag{8d}$$

$$\dot{x}_{m,i}(t) = f_i(x_i(t), u_i(t)), \quad t \in [t_{m-1}, t_m] \tag{8e}$$

$$R_i(x_{m,i}(t)) \cap R_j(x_{m,j}(t)) = \emptyset \quad i \neq j \tag{8f}$$

where J_m is the cost function defined in (5), (8c) decodes the

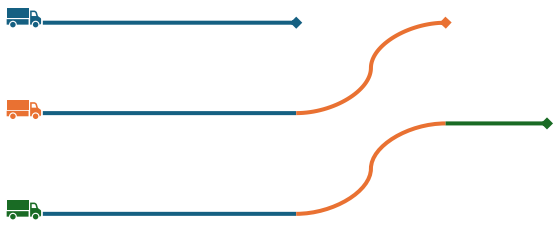

Fig. 2. Illustration of phases for three agents. Each color represents a different phase. Each phase covers the time until one of the agents reaches its goal. The initial positions are marked by the truck symbols, and the final positions are indicated by the arrowheads.

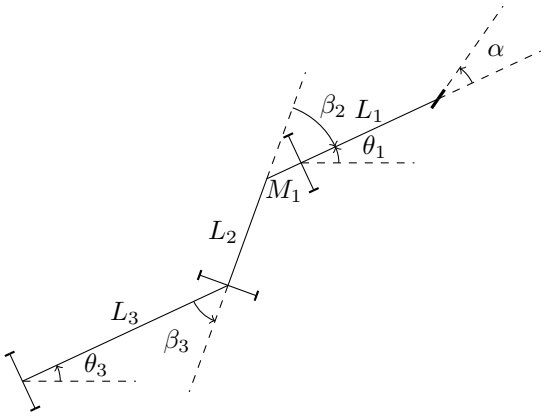

Fig. 3. An illustration of the truck and trailer system.

constraint that for each phase m the mth agent reaches its goal and constraint (8d) ensures continuity between phases.

VII. Numerical Results

In this section the proposed planning algorithms are implemented and evaluated in problems with truck and trailer systems in unstructured environments. The single-agent motion planner, CBS, and PBS planners are implemented in C++. The single-agent motion planner uses synchronized motion primitives that have been optimized as described in Section IV. It uses A* search guided by a heuristic function in the form of a heuristic lookup table (HLUT) [42] that has been precomputed offline. The HLUT contains the optimal costs of trajectories in obstacle-free space from all initial states $x_0 \in \mathcal{X}_d$ positioned at the origin to all final states $x_f \in \mathcal{X}_d$ positioned within a square centered around the origin with side length ρ. Note that the truck and trailer system is position and rotation invariant.

The computation of the motion primitives and the improvement step are both implemented in Python using CasADi [43] with IPOPT [44] and the ma57 solver from HSL.

A. Vehicle models

Each truck and trailer system is a general 2-trailer with a car-like truck and off-axle hitching [40], [45]. Each such system has three vehicle components: a car-like tractor, a dolly, and a semitrailer as illustrated in Figure 3. The state vector of the system is

$$\begin{aligned} \mathbf{x} &= \begin{bmatrix} \mathbf{z}^T & \alpha & \omega & v_1 & a_1 \end{bmatrix}^T \\ \mathbf{z} &= \begin{bmatrix} x_3 & y_3 & \theta_3 & \beta_3 & \beta_2 \end{bmatrix}^T \end{aligned} \tag{9}$$

where $\mathbf{z}$ describes the state of the truck and trailer, (x_3, y_3) is the position of the semitrailer, θ_3 is the orientation of the semitrailer, β_3 is the joint angle between the semitrailer and dolly and β_2 is the joint angle between the dolly and the truck. As in [46] the state vector has been augmented with α and ω, the steering angle and steering angle rate, respectively, of the truck as well as the longitudinal velocity v_1 and acceleration a_1 of the truck.

With the control signal vector

$$\mathbf{u} = \begin{bmatrix} u_\omega & u_a \end{bmatrix}^T \tag{10}$$

the system can be modelled as [46]:

$$\dot{\mathbf{x}} = \begin{bmatrix} v_1 f(\mathbf{z}, \alpha) & \omega & u_\omega & a_1 & u_a \end{bmatrix}^T \tag{11}$$

where

$$f(\mathbf{z}, \alpha) = \begin{bmatrix} \dot{x}_3 & \dot{y}_3 & \dot{\theta}_3 & \dot{\beta}_3 & \dot{\beta}_2 \end{bmatrix}^T \tag{12}$$

$$\begin{aligned} \dot{x}_3 &= \cos\beta_3 (1 + \frac{M_1}{L_1} \tan\beta_2 \tan\alpha) \cos\theta_3 \\ \dot{y}_3 &= \cos\beta_3 (1 + \frac{M_1}{L_1} \tan\beta_2 \tan\alpha) \sin\theta_3 \\ \dot{\theta}_3 &= \frac{\sin\beta_3 \cos\beta_2}{L_3} (1 + \frac{M_1}{L_1} \tan\beta_2 \tan\alpha) \\ \dot{\beta}_3 &= \cos\beta_2 (\frac{1}{L_2} (\tan\beta_2 - \frac{M_1}{L_1} \tan\alpha) - \\ &\qquad \frac{\sin\beta_3}{L_3} (1 + \frac{M_1}{L_1} \tan\beta_2 \tan\alpha)) \\ \dot{\beta}_2 &= \frac{\tan\alpha}{L_1} - \frac{\sin\beta_2}{L_2} + \frac{M_1}{L_1 L_2} \cos\beta_2 \tan\alpha \end{aligned} \tag{13}$$

The cost function, which is used both in the improvement step and during the search for the initial feasible solution, is selected as

$$l(\mathbf{x}, \mathbf{u}) = 1 + \frac{1}{2}(\alpha^2 + 10\omega^2 + a_1^2 + \mathbf{u}^T \mathbf{u}). \tag{14}$$

All agents have the same geometry as described in [40].

B. Simulation Results

To evaluate the performance of the proposed framework we first randomly generate 100 problem instances for $n = 2, 3, 4, 5$ agents in an obstacle-free 200×200 map. All problem instances are generated so that there is no overlap between initial positions or between terminal positions, and so that there exists a feasible trajectory for each agent if all other agents are ignored. Secondly, we randomly generate 100 additional problem instances in a 200×200 map where static obstacles are randomly generated as well. The HLUT radius is $\rho = 200$. We implemented both CBS and PBS with

TABLE I

COMPARISON OF THE AVERAGE PERFORMANCE OF THE PROPOSED SINGLE-AGENT PLANNERS.

Planner	Obstacle-free		Obstacles	
	Cost	Time [s]	Cost	Time [s]
Extended SIPP-IP	131.30	0.12	144.66	2.05
Lattice-based	130.80	0.008	126.19	3.54

both SIPP-IP and a lattice planner as single-agent planner using the same set of optimized motion primitives. Each planner was given up to 100 s to solve each problem instance.

We first compared SIPP-IP and the lattice-based planner by applying them to the generated problems as if they were single-agent problems. In total there were 500 such single-agent problems for the obstacle-free map and 500 such problems for the obstacle maps. The average computation times and plan costs are shown in Table I. It can be seen that the lattice-based planner is much faster than SIPP-IP when there are no obstacles present, but slower when obstacles are present. It can also be seen that both planners return nearly the same average cost without obstacles (the cost for SIPP-IP is slightly higher), but when obstacles are present the average cost is 14 % higher for SIPP-IP. This higher cost can be explained by the difference in collision avoidance between the planners. As the collision avoidance of SIPP-IP is more conservative, plans that are feasible for the lattice-based planner may be discarded as infeasible by SIPP-IP which is then forced to select a plan with higher cost instead. Note that the obstacle-free map is not truly obstacle-free as the boundaries of the map serve as obstacles, which explains the difference in cost for that map as well.

The reason that SIPP-IP is faster in the presence of obstacles is likely due to the efficient representation of time as intervals. SIPP-IP is therefore able to conclude for a whole time interval that applying a certain primitive from a certain state is impossible due to collision. The lattice-based planner on the other hand is only able to conclude that the primitive can not be applied at a given point in time and may therefore try to increasingly apply a wait primitive and then try the original primitive again.

Next, we applied the first step of the proposed multi-agent framework to the generated problems. Figure 4 shows the number of problem instances that were successfully solved by the planners within the allotted time of 100 s. It can be seen that for a given multi-agent planner the lattice-based planner always performed better than SIPP-IP. The difference in success rate can be explained by SIPP-IP having a more conservative collision avoidance, so that solutions that are feasible for the lattice-based planner are discarded as infeasible by SIPP-IP. This is the case both for static obstacles as well as dynamic obstacles arising from the trajectories of other agents. This can cause SIPP-IP to either fail to find a feasible solution, or to be so slow that it fails to find a feasible solution within the time limit. Similarly, we see that for a given single-agent planner PBS usually performs better than CBS, with the exception being in the case with obstacles for the lattice-based planner.

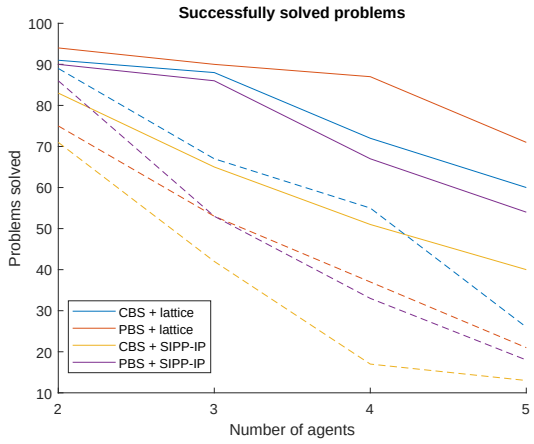

Fig. 4. The number of problems successfully solved by the first step within the allowed time limit. Fully drawn lines show the results for the obstacle-free map and dashed lines show the results for the maps with obstacles.

TABLE II

COMPARISON OF THE AVERAGE PERFORMANCE OF THE SIPP-IP AND THE LATTICE-BASED PLANNER AS SINGLE-AGENT PLANNERS FOR PBS CONSIDERING THE PROBLEMS BOTH PLANNERS MANAGED TO SOLVE.

Planner	Agents	Obstacle-free		Obstacles	
		Cost	T [s]	Cost	T [s]
SIPP-IP	2	265.71	2.10	288.58	7.95
Lattice-based	2	263.87	0.25	257.47	4.16
SIPP-IP	3	398.15	7.58	423.96	13.23
Lattice-based	3	393.16	1.48	376.27	5.98
SIPP-IP	4	521.31	5.31	579.38	37.50
Lattice-based	4	513.41	4.61	520.09	11.95
SIPP-IP	5	641.06	10.62	678.71	26.42
Lattice-based	5	626.86	7.29	602.48	16.52

In Table II a comparison for SIPP-IP and the lattice-based planners considering the problems that both planners managed to solve with PBS is shown. The lattice-based planner is faster than SIPP-IP in the obstacle-free case as well as when there are obstacles present. It also always performs better in terms of cost function. In the obstacle-free case the difference is only a few percent, but with obstacles present it is around 15 %.

For the evaluation of the proposed two-step MAMP approach we choose to use the lattice-based planner as it had a higher success rate. We evaluated both PBS and CBS as multi-agent planners. For all problems where a feasible solution was successfully found, we applied the continuous improvement step where the numerical solver was given 100 s to improve the solution.

Figure 5 shows the number of problems that were successfully solved by the planners. For the obstacle-free case, it can be seen that PBS solves a higher number of problems than CBS does and that the gap increases as the number of agents increases. It can also be seen that for $n = 2$ agents the optimization step is able to improve all initial

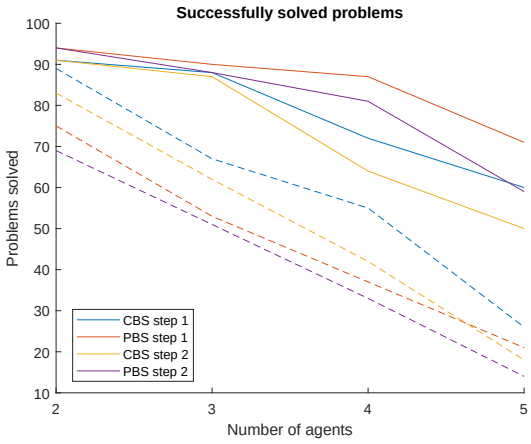

Fig. 5. The number of problems successfully solved by the two planners. Successfully solved during step 2 means that the numerical optimization solver was able to improve the solution. Fully drawn lines show the results for the obstacle-free map and dashed lines show the results for the maps with obstacles.

TABLE III

COMPARISON OF CBS AND PBS IN AN OBSTACLE-FREE ENVIRONMENT. PROBLEMS SOLVED BY BOTH PLANNERS. COMPUTATION TIME IS DENOTED WITH t, COST FUNCTION VALUE WITH J AND THE NUMBER OF PROBLEMS SOLVED WITHIN THE TIME LIMIT WITH r. SUBSCRIPTS 1 AND 2 DENOTE THE FIRST AND SECOND STEPS, RESPECTIVELY.

N	Planner	t_1	t_2	J_1	J_2	r_1	r_2
2	CBS	0.08	2.92	262.79	221.45	86	86
2	PBS	0.56	3.05	264.01	221.09	86	86
3	CBS	1.03	13.06	393.43	335.36	82	81
3	PBS	1.29	13.09	393.45	335.89	82	80
4	CBS	1.07	24.49	501.52	433.10	71	63
4	PBS	2.11	26.32	501.74	426.81	71	68
5	CBS	3.49	39.92	635.44	552.54	49	42
5	PBS	7.40	41.09	635.86	558.41	49	40

solutions, but as the number of agents increases the number of problems where an improved solution is found (within the time limit) decreases. The number of problems where a feasible solution is found during the first step also decreases, so the optimization step still succeeds for the majority of the problems where it is applied. Table III shows the results for the problems solved by both planners, to facilitate a direct comparison on execution time and cost function values. As seen in Table III, for the first step CBS is faster than PBS for all number of agents. As CBS is (resolution) optimal and PBS is not, it returns solutions with lower cost, but the difference in cost function value is very small. After the improvement step, however, the average cost is sometimes slightly lower for PBS and sometimes slightly lower for CBS. An example of resulting trajectories after the first and second steps is shown in Figure 6.

As seen in Figure 5, both planners solve less problems in the case with obstacles than for the obstacle-free case. Unlike the obstacle-free case, CBS performs better than PBS

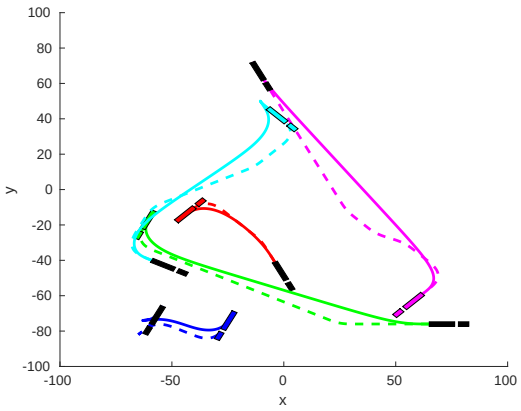

Fig. 6. Example of the resulting plans for an example with four agents. The black tractor-trailers show the initial positions and final positions are colored. Dashed lines show the trajectories after the first step and fully drawn lines show the final solution after the improvement step. It is clear that artefacts from discretization in the initial solution have been removed in the final solution.

TABLE IV

COMPARISON OF CBS AND PBS IN AN ENVIRONMENT WITH OBSTACLES FOR THE PROBLEMS SOLVED BY BOTH PLANNERS. COMPUTATION TIME IS DENOTED WITH t, COST FUNCTION VALUE WITH J AND THE NUMBER OF PROBLEMS SOLVED WITHIN THE TIME LIMIT WITH r. SUBSCRIPTS 1 AND 2 DENOTE THE FIRST AND SECOND STEPS, RESPECTIVELY.

N	Planner	t_1	$t_2 2$	J_1	J_2	r_1	r_2
2	CBS	1.68	3.26	251.59	211.21	75	69
2	PBS	4.21	3.27	251.67	210.68	75	69
3	CBS	1.57	14.01	372.52	314.22	49	47
3	PBS	6.48	13.61	372.54	314.27	49	47
4	CBS	3.98	23.38	499.45	422.38	32	28
4	PBS	6.13	22.36	499.67	421.03	32	28
5	CBS	11.77	45.25	583.41	497.76	13	10
5	PBS	16.86	39.36	583.94	506.69	13	9

for all number of agents. As in the obstacle-free case the number of problems where the solution is improved (within the time limit) during the improvement step decreases as the number of agents increases. A comparison between CBS and PBS is presented in Table IV where only the problems solved by both planners in the first step are considered. The time needed to find a solution has increased. CBS is still faster than PBS. As in the obstacle-free case, CBS finds solutions with a slightly lower average cost after the first step but after the improvement step PBS sometimes achieves solutions with slightly lower average cost. This is likely due to randomness rather than any inherent property of PBS, but it does demonstrate that a lower cost function value after the first step does not necessarily translate to a lower cost function value after the second step.

VIII. CONCLUSIONS AND FUTURE WORK

In this work we consider a multi-agent motion planning (MAMP) problem for tractor-trailers. The problem is challenging due to the complex kinematics of the vehicles as well as due to the vehicles being comprised of several connected bodies. We propose a framework for optimized MAMP

consisting of two steps. In the first step an initial, feasible and collision-free, solution is computed using conflict-based search (CBS) or priority-based search (PBS). The second step is an improvement step where a multi-phase optimal control problem (OCP) is posed and the solution found in the previous step is used to warm-start the solver.

We have also proposed a framework for automatically generating optimal motion primitives with the same time discretization, a requirement for the single-agent planner safe interval path planning with interval projection (SIPP-IP), in an optimized way. The cost function used to generate the primitives is consistently also the cost function used by the continuous improvement step.

We have implemented and evaluated the proposed framework on a large number of randomly generated problems for tractor-trailer systems. The results show that using a lattice-based planner as single-agent planner performs better than SIPP-IP, likely due to a less conservative collision avoidance. The results also show that although lacking completeness guarantees, PBS is able to solve a higher number of problems than CBS in an obstacle-free environment and can achieve solutions of similar quality after the improvement step. However, the results also show that CBS is faster than PBS on the problems they both solve and solves a higher number of problems in environments with obstacles.

Future work includes improving the computational efficiency of the implementation of the improvement step by using distributed optimization and parallelization. Another possible line of research is the addition of traffic rules that the agents should obey.

REFERENCES

[1] R. Stern, N. Sturtevant, A. Felner, S. Koenig, H. Ma, T. Walker, J. Li, D. Atzmon, L. Cohen, T. Kumar, *et al.*, "Multi-agent pathfinding: Definitions, variants, and benchmarks," in *Proceedings of the International Symposium on Combinatorial Search*, vol. 10, pp. 151–158, 2019.

[2] R. Stern, "Multi-agent path finding–an overview," *Artificial Intelligence: 5th RAAI Summer School, Dolgoprudny, Russia, July 4–7, 2019, Tutorial Lectures*, pp. 96–115, 2019.

[3] J. Gao, Y. Li, X. Li, K. Yan, K. Lin, and X. Wu, "A review of graph-based multi-agent pathfinding solvers: From classical to beyond classical," *Knowledge-Based Systems*, p. 111121, 2023.

[4] J. Yu and S. LaValle, "Structure and intractability of optimal multi-robot path planning on graphs," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 27, pp. 1443–1449, 2013.

[5] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial intelligence*, vol. 219, pp. 40–66, 2015.

[6] E. Boyarski, A. Felner, R. Stern, G. Sharon, O. Betzalel, D. Tolpin, and E. Shimony, "ICBS: The improved conflict-based search algorithm for multi-agent pathfinding," in *Proceedings of the International Symposium on Combinatorial Search*, vol. 6, pp. 223–225, 2015.

[7] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[8] T. Standley, "Finding optimal solutions to cooperative pathfinding problems," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 24, pp. 173–178, 2010.

[9] M. Goldenberg, A. Felner, R. Stern, G. Sharon, N. Sturtevant, R. C. Holte, and J. Schaeffer, "Enhanced partial expansion A*," *Journal of Artificial Intelligence Research*, vol. 50, pp. 141–187, 2014.

[10] G. Wagner and H. Choset, "Subdimensional expansion for multirobot path planning," *Artificial intelligence*, vol. 219, pp. 1–24, 2015.

[11] G. Sharon, R. Stern, M. Goldenberg, and A. Felner, "The increasing cost tree search for optimal multi-agent pathfinding," *Artificial intelligence*, vol. 195, pp. 470–495, 2013.

[12] T. T. Walker, N. R. Sturtevant, and A. Felner, "Extended Increasing Cost Tree Search for Non-Unit Cost Domains.," in *IJCAI*, pp. 534–540, 2018.

[13] M. Erdmann and T. Lozano-Perez, "On multiple moving objects," *Algorithmica*, vol. 2, pp. 477–521, 1987.

[14] J. P. van den Berg and M. H. Overmars, "Prioritized motion planning for multiple robots," in *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 430–435, 2005.

[15] D. Silver, "Cooperative pathfinding," in *Proceedings of the AAAI conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 1, pp. 117–122, 2005.

[16] H. Ma, D. Harabor, P. J. Stuckey, J. Li, and S. Koenig, "Searching with consistent prioritization for multi-agent path finding," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, pp. 7643–7650, 2019.

[17] D. Atzmon, A. Diei, and D. Rave, "Multi-train path finding," in *Proceedings of the International Symposium on Combinatorial Search*, vol. 10, pp. 125–129, 2019.

[18] J. Li, P. Surynek, A. Felner, H. Ma, T. S. Kumar, and S. Koenig, "Multi-agent path finding for large agents," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, pp. 7627–7634, 2019.

[19] L. Cohen, T. Uras, T. Kumar, and S. Koenig, "Optimal and bounded-suboptimal multi-agent motion planning," in *Proceedings of the International Symposium on Combinatorial Search*, vol. 10, pp. 44–51, 2019.

[20] A. Andreychuk, K. Yakovlev, P. Surynek, D. Atzmon, and R. Stern, "Multi-agent pathfinding with continuous time," *Artificial Intelligence*, vol. 305, p. 103662, 2022.

[21] M. Phillips and M. Likhachev, "SIPP: Safe interval path planning for dynamic environments," in *2011 IEEE international conference on robotics and automation*, pp. 5628–5635, IEEE, 2011.

[22] Z. A. Ali and K. Yakovlev, "Safe interval path planning with kinodynamic constraints," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, pp. 12330–12337, 2023.

[23] L. Wen, Y. Liu, and H. Li, "CL-MAPF: Multi-agent path finding for car-like robots with kinematic and spatiotemporal constraints," *Robotics and Autonomous Systems*, vol. 150, p. 103997, 2022.

[24] J. Kottinger, S. Almagor, and M. Lahijanian, "Conflict-based search for multi-robot motion planning with kinodynamic constraints," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 13494–13499, IEEE, 2022.

[25] L. Su, M. Yue, X. Sun, H. Wang, and X. Zhao, "Collaborative Motion Planning for Multiple Tractor-Trailer Vehicles Based on Local Conflict Search and Priority Game Inheritance," *IEEE Robotics and Automation Letters*, 2025.

[26] I. Solis, J. Motes, R. Sandström, and N. M. Amato, "Representation-Optimal Multi-Robot Motion Planning Using Conflict-Based Search," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4608–4615, 2021.

[27] S. LaValle, "Rapidly-exploring random trees: A new tool for path planning," *Research Report 9811*, 1998.

[28] S. M. LaValle and J. J. Kuffner Jr, "Randomized kinodynamic planning," *The international journal of robotics research*, vol. 20, no. 5, pp. 378–400, 2001.

[29] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The international journal of robotics research*, vol. 30, no. 7, pp. 846–894, 2011.

[30] S. Karaman and E. Frazzoli, "Sampling-based optimal motion planning for non-holonomic dynamical systems," in *2013 IEEE international conference on robotics and automation*, pp. 5041–5047, IEEE, 2013.

[31] S. Stoneman and R. Lampariello, "Embedding nonlinear optimization in RRT* for optimal kinodynamic planning," in *53rd IEEE Conference on Decision and Control*, pp. 3737–3744, IEEE, 2014.

[32] J. Yan and J. Li, "Multi-Agent Motion Planning With Bézier Curve Optimization Under Kinodynamic Constraints," *IEEE Robotics and Automation Letters*, 2024.

[33] B. Li, T. Acarman, Y. Zhang, L. Zhang, C. Yaman, and Q. Kong, "Tractor-trailer vehicle trajectory planning in narrow environments with a progressively constrained optimal control approach," *IEEE Transactions on Intelligent Vehicles*, vol. 5, no. 3, pp. 414–425, 2019.

[34] K. Bergman, O. Ljungqvist, and D. Axehill, "Improved path planning by tightly combining lattice-based path planning and optimal control,"

IEEE Transactions on Intelligent Vehicles, vol. 6, no. 1, pp. 57–66, 2020.

[35] M. Pivtoraiko, R. A. Knepper, and A. Kelly, "Differentially constrained mobile robot motion planning in state lattices," *Journal of Field Robotics*, vol. 26, no. 3, pp. 308–333, 2009.

[36] J. Dahlmann, A. Völz, M. Lukassek, and K. Graichen, "Local predictive optimization of globally planned motions for truck-trailer systems," *IEEE Transactions on Control Systems Technology*, 2023.

[37] C. Toumieh and A. Lambert, "Decentralized Multi-Agent Planning Using Model Predictive Control and Time-Aware Safe Corridors," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 11110–11117, 2022.

[38] H. Li, P. Chen, G. Yu, B. Zhou, Z. Han, and Y. Liao, "Collaborative Trajectory Planning for Autonomous Mining Trucks: A Grouping and Prioritized Optimization Based Approach," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 5, pp. 6283–6300, 2024.

[39] S. M. LaValle, *Planning algorithms*. Cambridge university press, 2006.

[40] O. Ljungqvist, N. Evestedt, D. Axehill, M. Cirillo, and H. Pettersson, "A path planning and path-following control framework for a general 2-trailer with a car-like tractor," *Journal of field robotics*, vol. 36, no. 8, pp. 1345–1377, 2019.

[41] K. Bergman, O. Ljungqvist, and D. Axehill, "Improved optimization of motion primitives for motion planning in state lattices," in *2019 IEEE intelligent vehicles symposium (IV)*, pp. 2307–2314, IEEE, 2019.

[42] R. A. Knepper and A. Kelly, "High Performance State Lattice Planning Using Heuristic Look-Up Tables.," in *IROS*, pp. 3375–3380, Citeseer, 2006.

[43] J. A. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi: a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, pp. 1–36, 2019.

[44] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical programming*, vol. 106, pp. 25–57, 2006.

[45] C. Altafini, A. Speranzon, and K. H. Johansson, "Hybrid control of a truck and trailer vehicle," in *International Workshop on Hybrid Systems: Computation and Control*, pp. 21–34, Springer, 2002.

[46] K. Bergman, O. Ljungqvist, T. Glad, and D. Axehill, "An optimization-based receding horizon trajectory planning algorithm," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 15550–15557, 2020.